

Quiz 1

Reference:

```
public interface Position<E> {  
    E getElement();  
}
```

```
public interface PositionalList<E> {  
    int size();  
    boolean isEmpty();  
    Position<E> first();  
    Position<E> last();  
    Position<E> before(Position<E> p);  
    Position<E> after(Position<E> p);  
    Position<E> addFirst(E e);  
    Position<E> addLast(E e);  
    Position<E> addBefore(Position<E> p, E e);  
    Position<E> addAfter(Position<E> p, E e);  
    E set(Position<E> p, E e);  
    E remove(Position<E> p);  
}
```

Question. Write a method that checks if two elements **a** and **b** exist anywhere separated by 2 spots in a list:

```
public static <E> boolean gapTwoSearch(E a, E b, PositionalList<E> list)
```

For example, given a list with elements [4, 6, 3, 2]:

- `gapTwoSearch(4, 2, list)` returns `true`
- `gapTwoSearch(2, 4, list)` returns `true`
- `gapTwoSearch(4, 3, list)` returns `false`

Your method must be efficient.

Solution

```
public static <E> boolean gapTwoSearch(E a, E b, PositionalList<E> list) {
    Position<E> current = list.first();
    while (current != null) {
        Position<E> gapAhead = list.after(current);           // 1 ahead
        if (gapAhead != null) gapAhead = list.after(gapAhead); // 2 ahead
        if (gapAhead != null) gapAhead = list.after(gapAhead); // 3 ahead
        if (gapAhead != null) {
            E currentElem = current.getElement();
            E aheadElem = gapAhead.getElement();
            if ((currentElem.equals(a) && aheadElem.equals(b)) ||
                (currentElem.equals(b) && aheadElem.equals(a)))
                return true;
        }
        current = list.after(current);
    }
    return false;
}
```

Walk through the list, and at each position check if it and the position three ahead form the pair (a, b) in either order. $O(n)$ time.